

technical stack scope for an open-source eeg and meg workflow and repository platform

executive summary

the right mental model for a synaptix or gliahub-like product is not “a workflow editor with some neuroscience support”, but “a reproducible neuroscience operating layer” built around three first-class objects: datasets, workflows, and executions. for eeg and meg first, the safest canonical format is bids for dataset packaging and discovery, with nwb as a complementary analysis and exchange format rather than the only internal truth. bids is already the organising standard behind openneuro, and dandi explicitly centres nwb and related conversion tooling in its archive workflows. that means an mvp should treat bids as the primary repository contract, while exposing import and export paths to nwb through pynwb, matnwb, neuroconv, and related validators. ¹

for the domain stack, mne-python should be the centre of gravity. it is the broadest open-source python toolkit in this space for eeg, meg, ecog, seeg, nirs and associated analysis workflows; braindecode sits naturally on top for deep learning on raw electrophysiology; moabb gives you benchmarkable eeg pipelines and dataset abstractions; pybids and the bids validator should sit in the ingest and validation path; eeglab and brainstorm are important interoperability targets, not libraries to reimplement inside the core runtime. nilearn matters if you expect the product to grow toward source-space neuroimaging, surface/statistical maps, or multimodal work later. torch EEG is useful where you want a faster “plug-and-play” deep-learning experience for eeg specifically, but it is less foundational than mne plus braindecode plus moabb for a platform substrate. ²

for execution, containers are the default answer. the platform should package every executable node as an immutable container image, schedule runs on kubernetes, and persist artefacts to s3-compatible object storage. among workflow engines, argo workflows is the best fit for a product that wants hosted execution, batch or gpu jobs, k8s-native control, and a clear separation between your application layer and runtime layer. nextflow is the strongest companion technology for import and export of portable scientific pipelines and for future hpc support, but it is a weaker primary engine for an app-centric visual product. airflow is mature but too etl-centric for this use case; prefect is very ergonomic for python-first teams; dagster is compelling if you want stronger lineage and asset semantics, but it is less naturally aligned than argo to multi-tenant container execution of arbitrary scientific jobs. ³

the best mvp architecture is therefore: react or next.js frontend with react flow for the visual editor; a python backend with a clean rest api and a python sdk; postgresql for relational metadata and snapshot manifests; object storage for datasets and artefacts; opensearch only if search requirements outgrow postgres full-text search; argo workflows on kubernetes for execution; containerd as the default runtime; keycloak or a similar idp layer for oidc and saml; append-only audit events; and signed container builds with sigstore and slsa provenance. for a two-year roadmap, add openlineage-based lineage, gvisor or equivalent hardened sandboxes for untrusted multi-tenancy, nextflow interoperability, remote kernels for notebook-like interactive work, and optional wasm-based micro-sandboxes for lightweight validation and transformation steps. ⁴

scope and assumptions

this report assumes the platform is open source at the application layer, targets eeg and meg first, must support both self-hosted and managed saas deployment, and needs to serve three user groups: individual researchers, institutional labs, and regulated or semi-regulated organisations that may eventually require stronger data governance. it also assumes that “similar to synaptix or gliahub” means the product surface includes a visual editor, python export, executable and repeatable pipelines, repository and snapshot semantics, private collaboration spaces, and some level of hosted execution rather than a pure client-side notebook tool. where current public documentation is silent or fragmented, i mark recommendations as architectural judgement rather than settled fact. ⁵

a second assumption is that raw scientific data should not be stored with git-like semantics in the same way as source code. openneuro exposes dataset snapshots and uses datalad for distribution, while pennsieve presents dataset-version and workspace concepts on top of cloud storage and metadata services. that points to the same conclusion for a new platform: version metadata, workflow definitions, and manifests like source code; version large signal files as immutable blobs plus snapshot manifests. “repo/versioning” in this domain should therefore mean reproducible dataset states, workflow states, and execution outputs, not direct git storage of multi-gigabyte eeg or meg artefacts. ⁶

the final assumption is that bids and nwb are not competitors in the way product roadmaps sometimes frame them. bids is an organisation and metadata standard that already anchors repositories such as openneuro and a large validation and python tooling ecosystem; nwb is a unified neurophysiology data format with strong archive and tooling support through dandi, pynwb, and matnwb. in practice, a serious eeg and meg platform should support both, but with a clear internal bias: bids for repository ingest, packaging, and discoverability; nwb for rich data interchange, downstream archives, derived outputs, or modality-specific workflows that benefit from nwb-native tooling. ⁷

product surface and domain stack

the core product features split naturally into two layers. the first is the user-facing product layer: visual workflow editor, execution runs, snapshotting, workspace sharing, access control, auditability, and python export. the second is the scientific substrate: bids and nwb awareness, neurophysiology libraries, format validators, and tool wrappers for matlab-based ecosystems. the platform only feels credible if both layers are strong. openneuro’s public positioning around bids validation, private sharing, and snapshots, and pennsieve’s positioning around workspaces, roles, analysis, metadata schemas, and publication show that repository products in this category win on workflow-plus-governance rather than compute alone. ⁶

for the scientific substrate, mne-python is the anchor choice for an eeg and meg-first platform. its current documentation explicitly covers exploration, visualisation, and analysis across meg, eeg, seeg, ecog, nirs and related methods, making it the broadest python-native base for shared data loading, pre-processing, spectral analysis, connectivity, source estimation, and decoding. mvp node types should therefore wrap mne primitives first: reading and writing common eeg or meg formats, resampling, filtering, rejection, epoching, covariance, time-frequency, source-space transforms, and export adapters. this yields immediate scientific legitimacy without forcing the product team to invent file readers or signal operations from scratch. ⁸

braindecode is the best deep-learning companion when the platform needs trainable models on raw electrophysiology. its documentation describes it as a pytorch-native toolbox for end-to-end neural decoding of eeg, ecog, meg, emg and ieeg, and it explicitly integrates with mne and moabb. that makes

it attractive for executable training nodes, benchmark recipes, and model-zoo-backed examples. moabb is equally important, not because it should own your runtime, but because it standardises datasets, paradigms, evaluations, and pipelines for eeg benchmarking. together, mne plus braindecode plus moabb give you a credible path from raw signals to reproducible baselines and future model sharing.

9

pybids and the bids validator belong in the ingestion path, not as optional extras. pybids gives you programmatic querying of bids datasets; the validator is now available as a web app, cli, and javascript or typescript library, and the web validator explicitly performs validation locally in the browser without uploading files. that means the platform can support both client-side preflight validation and server-side admission checks before a dataset ever enters execution. for an eeg and meg product, this is a major trust feature: users should see dataset admissibility before they schedule compute. 10

nwb support should be deliberate and narrow at first. pynwb is the python reference layer for reading and writing nwb; matnwb gives the matlab side; and neuroconv is currently the cleanest conversion package for taking a wide variety of proprietary neurophysiology formats into nwb with attention to metadata extraction, chunking, compression, temporal alignment, and large datasets. that combination argues for an mvp where nwb is supported mainly through import, export, archive hand-off, and selected readers or writers, rather than as the primary repository abstraction from day one. it also provides a clean interop path to dandi, whose documentation explicitly exposes both a rest api and a client. 11

eeglab and brainstorm should be treated as integration targets and workflow nodes, not as internal platform runtimes. eeglab's own documentation emphasises its long-standing role as a widely used environment with an extension and plugin ecosystem, including eeg-bids and nwb-io plugins. brainstorm's official documentation emphasises its gui-first, user-friendly analysis environment, batch scripting, broad modality support, supported file formats including bids datasets and nwb, and compiled distribution that does not require a matlab licence. in a product strategy sense, both are too important to ignore and too large to re-implement. the right move is to make them first-class adapters: import pipeline steps, export pipeline steps, containerised node templates where feasible, and migration helpers that preserve users' existing lab habits. 12

nilearn is not essential for an eeg or meg-only mvp, but it becomes strategically useful if the platform wants to grow into source-space maps, group statistics, surface visualisation, or multimodal neuroimaging. nilearn's official positioning is around approachable statistical and machine-learning analysis of brain volumes and surfaces, including glm, connectivity, decoding, and connectomics. if the roadmap includes fmri, mri-linked source workflows, or richer statistical report generation, nilearn should move from "optional" to "roadmap core". torcheeg is the opposite: useful now for eeg-specific deep-learning ergonomics, but not a repository or execution-platform cornerstone. 13

the practical recommendation is therefore straightforward. for the mvp, make mne, pybids, bids validator, braindecode, and moabb the required stack; keep neuroconv and pynwb as first-party import or export paths; expose eeglab and brainstorm via adapters; and defer deeper nilearn support unless product scope is already broadening beyond eeg and meg. for the two-year roadmap, add richer nwb-native workflows, atlas and source-space features, and a plugin interface that lets the community contribute node wrappers around the broader neuroscience ecosystem. 14

platform architecture and infrastructure

the architecture should be data-heavy, metadata-rich, and execution-decoupled. raw and derived datasets belong in object storage; relational metadata, permissions, manifests, and run state belong in postgresql; search can start in postgres and only move to opensearch if cross-workspace full-text or faceted metadata search becomes materially complex. postgres already gives strong jsonb support and built-in full-text search, which is enough for most early repository metadata use cases. opensearch becomes justified when users expect sub-second text search across very large metadata catalogues, vector search, or a more dedicated search service boundary. ¹⁵

for object storage, s3-compatible storage is the right abstraction line. on cloud, native s3-class services remain the simplest operational default; on-prem or in private clusters, minio gives a direct s3-compatible option. the reason this matters is less branding than compatibility: every execution engine, python workflow, and archive connector in this space already expects object semantics. scientific datasets are also a natural fit for object storage because they are large, append-light, and consumed by batch jobs much more often than they are mutated in place. ¹⁶

for execution orchestration, kubernetes is the right base platform when hosted execution is on the product promise. the official kubernetes overview is blunt about what it gives you: scheduling, service discovery, storage orchestration, rollouts, self-healing, secrets handling, batch execution, and horizontal scaling. these are precisely the primitives a multi-user compute platform needs. it is not the whole platform, but it is the best substrate when workloads are containerised and bursty. for self-hosted users, it also preserves deployment symmetry between local clusters, managed clusters, and regulated environments. ¹⁷

among workflow engines, argo workflows is the strongest primary choice for this product category because it is explicitly a workflow engine for kubernetes and assumes a cluster-backed, container-native world. its fit is especially good when each workflow node can be represented as a container image, when runs need to be isolated, and when gpu or batch-heavy scheduling matters. nextflow, by contrast, is excellent at portable scientific workflows and is explicitly designed to run across local, cloud, and hpc environments while describing dependencies between processes and artifacts. that makes it a strong interoperability target and perhaps a second runtime class for institutions, but not the best fit for an application-defined visual product mvp. airflow is extremely mature for batch-oriented workflows, but its own documentation frames it as a platform for developing, scheduling, and monitoring batch-oriented workflows, which is a narrower match than argo for this use case. prefect is more python-native and lower-friction, and dagster is notably strong on assets, observability, and testability; both are valid alternatives for smaller teams, but neither beats argo on “hosted scientific execution over k8s” as the dominant requirement. ¹⁸

for execution sandboxes, containers should be the default, gvisor should be the hardening path, and wasm should be limited to lightweight plugins for now. containerd is an industry-standard container runtime designed for image transfer, container execution, supervision, and related lifecycle concerns; that makes it the least controversial base runtime in a k8s-heavy platform. gvisor provides a stronger isolation layer by inserting an application kernel between workloads and the host operating system, which is compelling once you host less-trusted user code. wasmtime and the broader wasm ecosystem are technically attractive for untrusted extensions, but the current neuroscience python stack is not naturally wasm-native, and experimental k8s wasm projects like krustlet remain explicitly experimental. the implication is simple: scientific jobs stay in containers; tiny validation, parsing, or stateless transform steps may later move to wasm. remote kernels, meanwhile, are useful for interactive notebooks but make a poor default trust boundary for a multi-tenant execution service. jupyter

enterprise gateway is proof that remote-kernel style architectures are powerful, but it also reminds you that they are part of the attack surface and should be introduced carefully. 19

on the frontend, react is the obvious baseline and next.js is a pragmatic full-stack wrapper when you want built-in routing, server and client components, and a faster path to product delivery. for the visual editor, react flow is the best starting point because it is explicitly built around interactive flow graphs with custom nodes, handles, edges, minimaps, controls, and panels. if the product later needs denser graph analysis or network-centric views rather than node-editor ergonomics, cytoscape.js is the stronger companion library: it is a deeply featured graph visualisation and analysis library with layouts, graph algorithms, selectors, and production use in research and industry. for real-time collaboration, yjs is the open foundational choice because it is a high-performance crdt for collaborative apps, while a hosted collaboration layer such as liveblocks can accelerate delivery if the team accepts an external dependency. the safe mvp choice is react plus react flow plus yjs, with liveblocks as an optional acceleration rather than a hard dependency. 20

for apis, the platform should be rest-first and python-sdk-first. the reason is bluntly practical: your users, your scientific wrappers, and most of your community extension surface will be python-heavy. a clean rest surface makes browser integration, self-hosted ops, and public docs easier. gRPC is still useful for internal service-to-service calls or high-throughput control-plane interactions because it is designed for high-performance rpc with streaming and rich code generation, but it should stay inside the platform boundary until real performance bottlenecks demand public exposure. the python sdk should feel like a stable scientific client: construct datasets, validate them, register workflows, launch runs, stream logs, fetch artefacts, and export to notebook-friendly code. 21

the architecture below reflects those choices.

```
flowchart tb
    subgraph self_hosted["self-hosted deployment"]
        sh_users["researchers / lab admins"]
        sh_ui["web app\nreact or next.js + react flow"]
        sh_api["api layer\npython service + rest + python sdk"]
        sh_auth["iam\nkeycloak or similar\noidc / saml"]
        sh_db["postgresql\nmetadata, manifests, permissions, runs"]
        sh_search["optional opensearch\ncatalog and faceted search"]
        sh_obj["s3-compatible object storage\nminio or cloud bucket"]
        sh_orch["kubernetes + argo workflows"]
        sh_rt["container runtime\ncontainerd\noptional gvisor"]
        sh_workers["cpu / gpu workers\nscientific containers"]
        sh_ingest["ingest / validation\npybids + bids validator + neuroconv"]
        sh_libs["analysis stack\nmne, braindecode, moabb,\nnpynwb, eeglab/brainstorm adapters"]
        sh_logs["audit / telemetry\nappend-only events + lineage"]
    end

    subgraph saas["managed saas deployment"]
        saas_users["tenant users / org admins"]
        saas_ui["multi-tenant web app"]
        saas_api["control plane api"]
        saas_auth["enterprise sso\noidc / saml / scim later"]
        saas_db["postgresql\nmulti-tenant metadata"]
```

```

saas_search["optional opensearch"]
saas_obj["managed object storage"]
saas_orch["managed kubernetes + argo"]
saas_rt["containerd\noptional hardened sandboxes"]
saas_workers["ephemeral batch workers\ncpu / gpu"]
saas_logs["central audit, metrics,\nopenlineage later"]
end

sh_users --> sh_ui --> sh_api
sh_api --> sh_auth
sh_api --> sh_db
sh_api --> sh_search
sh_api --> sh_obj
sh_api --> sh_orch --> sh_rt --> sh_workers
sh_api --> sh_ingest --> sh_libs
sh_workers --> sh_obj
sh_workers --> sh_logs
sh_api --> sh_logs

saas_users --> saas_ui --> saas_api
saas_api --> saas_auth
saas_api --> saas_db
saas_api --> saas_search
saas_api --> saas_obj
saas_api --> saas_orch --> saas_rt --> saas_workers
saas_workers --> saas_obj
saas_workers --> saas_logs
saas_api --> saas_logs

```

comparison of candidate technologies

the table below compares the most plausible choices for the key layers that matter most to this kind of platform. “maturity” is an evaluative judgement based on project age, ecosystem breadth, and how closely the documented project model matches the platform’s needs; where a managed cloud service is not open source, i mark it as proprietary. the mvp recommendation column answers a narrow question: “is this the best default first implementation for an eeg and meg workflow repository product?”, not “is this technology good in general?”. ²²

tech	maturity	license	pros	cons	recommended for mvp?
workflow engine: argo workflows	high	apache-2.0	k8s-native, container-first, strong fit for hosted batch and gpu jobs, clean separation between product and runtime	assumes kubernetes competence; product team must build its own UX on top	yes

tech	maturity	license	pros	cons	recommended for mvp?
workflow engine: nextflow	high	apache-2.0	excellent reproducibility and portability across cloud and hpc; natural fit for imported scientific pipelines	weaker fit as the primary embedded execution engine behind a web product	no, but yes for roadmap interop
workflow engine: airflow	very high	apache-2.0	battle-tested, huge ecosystem, familiar to many ops teams	batch and etl-centric, heavier than necessary for interactive scientific product execution	no
workflow engine: prefect	medium-high	apache-2.0	very pythonic, low friction, good for smaller teams and code-first workflows	less naturally aligned than argo to container-native multi-tenant platform execution	maybe, if team is python-first and k8s-light
workflow engine: dagster	high	apache-2.0	strong observability, lineage-oriented model, good testability	more data-asset-oriented than scientific compute platform oriented	maybe, if metadata lineage is the dominant product axis
orchestration: kubernetes	very high	apache-2.0	standard substrate for containers, scaling, secrets, batch, storage integration	operational complexity, especially for self-hosted small labs	yes
orchestration: aws batch style managed batch	high	proprietary	removes much of the cluster management burden for batch workloads	cloud lock-in; harder to provide cloud-agnostic self-hosted parity	no as primary default

tech	maturity	license	pros	cons	recommended for mvp?
orchestration: cloud run jobs style serverless batch	medium-high	proprietary	zero-idle control model for some batch tasks	not ideal for heavier scientific pipelines, broad hpc portability, or complex gpu scheduling	no as primary default
storage: s3-class object storage	very high	proprietary service model	ecosystem standard, simple mental model, ideal for artefacts and datasets	egress can become expensive; some providers charge significantly for retrieval or requests	yes
storage: minio	high	agplv3 open-source edition	strong s3 compatibility, good self-hosted option, avoids cloud dependence	self-hosted ops burden; enterprise features and support model may matter at scale	yes for self-hosted
db: postgresql	very high	postgresql license	reliable relational core, jsonb, full-text search, transactional integrity	not the best dedicated search engine for very large catalogues	yes
db/search: opensearch	high	apache-2.0	dedicated search and analytics engine, stronger faceting and search features	extra infra and indexing complexity; premature for many mvps	only if search truly demands it
frontend graph lib: react flow	high	mit	best ergonomics for node editors, custom nodes, controls, minimap, panels	less suited to dense analytical graph visualisation	yes

tech	maturity	license	pros	cons	recommended for mvp?
frontend graph lib: cytoscape.js	high	mit	deep graph analysis and visualisation, rich layouts and algorithms	more work to make it feel like a polished node editor	no as the primary editor
container runtime: containerd	very high	apache-2.0	standard runtime, strong portability, aligned with k8s	lower-level, less product-facing abstractions	yes
container runtime: gvisor	medium-high	apache-2.0	stronger isolation for multi-tenant hosted execution	extra operational and performance trade-offs; not needed on day one for every deployment	roadmap
container runtime: podman	high	apache-2.0	daemonless and attractive for local dev or some self-hosted setups	less natural than containerd as the default runtime under k8s	local dev only

security, compliance, deployment models and economics

for security and compliance, the product needs to be designed as if it will eventually hold personal data, not just public datasets. the hipaa security rule requires administrative, physical, and technical safeguards for electronic protected health information. gdpr article 32 similarly requires technical and organisational measures appropriate to risk, explicitly naming pseudonymisation, encryption, confidentiality, integrity, availability, resilience, restoration, and regular testing. the consequence is that even if the initial product is targeted at open or lightly governed research, its architecture should already separate identities, enforce workspace boundaries, encrypt data in transit and at rest, and maintain durable audit evidence of who accessed or executed what. ²³

enterprise authentication should be standards-based from the start. saml remains a common enterprise federation mechanism and is explicitly designed to exchange authentication, attribute, and authorisation assertions between security domains. openid connect is the more developer-friendly modern identity layer on top of oauth 2.0. projects such as zitadel and keycloak demonstrate that open-source identity infrastructure can expose oidc and saml, support multi-tenancy, and provide auditable identity events. for this platform, the correct mvp move is not to build identity, but to integrate a proven idp layer and keep internal authorization simple and explicit: organisation → workspace → project → dataset/workflow/run. ²⁴

rbac should be coarse at first and domain-aware rather than clever. auth0's documentation captures the basic operational logic: assign permissions to roles, not directly to users. in this product, the minimal role set is usually enough: organisation admin, workspace manager, contributor, runner, reviewer, and read-only. resource-level grants can come later. a common mistake in scientific products is to overdesign permissions early and end up with a system nobody can reason about. better to keep the core matrix simple, expose it in the ui, and record every escalation, membership change, dataset publication, and execution action in immutable audit trails. ²⁵

for deployment models, three modes are worth supporting. self-hosted is essential for labs and institutions with data residency or network constraints. managed saas is essential if the product wants fast onboarding and a sustainable commercial path. hybrid matters once institutions want a hosted control plane but local or vpc-bound execution. because kubernetes and s3-compatible storage abstract the execution and data layers, a well-designed control plane can support all three without changing the platform contract much. pennsieve's cloud-centric design and openneuro's repository patterns show why this matters: the product needs enough control-plane consistency that repositories, snapshots, and collaboration feel the same even when compute placement changes. ²⁶

the cost structure is also unusually specific in this category. storage is rarely the only problem; egress and idle compute often dominate once users start moving large signals around or keeping gpu workers warm. public pricing documents make the storage point clear enough: amazon s3 standard starts at about \$0.023 per gb-month for the first 50 tb, while cloudflare r2 standard storage is listed at \$0.015 per gb-month with free direct egress from r2. on storage alone, 10 tb of frequently accessed data therefore lands around £150–£230 per month equivalent before request costs and before regional taxes or discounts, depending on provider and class. the bigger risk is poor architecture: if every visual interaction re-downloads large artefacts, egress outstrips storage very quickly. ²⁷

for compute, exact monthly numbers depend too much on job mix, but the structural trade-off is stable. on-demand or always-on clusters translate fixed hourly rates into persistent baseline cost; managed batch and serverless models reduce idle waste by provisioning only in response to jobs. aws batch explicitly positions itself around automatic provisioning and workload distribution for batch jobs at scale, while serverless batch platforms emphasise “pay only for actual compute time” and scale-to-zero behaviour. the recommendation follows directly: keep the control plane lean and persistent, but make scientific workers ephemeral; reserve always-on gpu nodes only when utilisation is consistently high or queue-time requirements are strict. ²⁸

a realistic mvp monthly envelope, excluding headcount, usually looks like this in structural terms rather than one fake precise number: a small persistent control plane, one managed relational database, object storage, modest logging and metrics, and bursty cpu workers. that is often acceptable for an early-stage saas or institutional pilot. the cost curve steepens once you add high-retention logs, opensearch clusters, always-on collaboration infrastructure, and especially persistent gpu capacity. the strongest economic recommendation is therefore architectural, not just financial: use object storage heavily, use compute ephemerally, avoid premature dedicated search clusters, and make egress visible to users and admins from the start. ²⁹

recommended stack and roadmap

for the mvp, i would choose a deliberately conservative stack. the frontend should be react or next.js with react flow as the workflow canvas and yjs for collaborative state. the backend should be a python service that exposes a stable rest api and ships a first-party python sdk. postgres should hold users, workspaces, dataset metadata, snapshot manifests, workflow definitions, run state, access grants, and

audit events. datasets and artefacts should live in s3-compatible object storage. execution should run through argo workflows on kubernetes, with containerd underneath. the scientific core should be mne, pybids, the bids validator, braindecode, and moabb; pynwb and neuroconv should handle nwb and conversion paths; eeglab and brainstorm should be supported through file and execution adapters, not by embedding matlab assumptions into the main platform. auth should be delegated to keycloak or an equivalent oidc and saml layer. build artefacts should be stored in an oci registry and signed with sigstore-style signing, with salsa provenance attached where practical. ³⁰

that mvp should intentionally cut scope in a few places. do not start with a fully general collaborative notebook product. do not build a custom search engine before postgres indexing is clearly insufficient. do not treat git semantics for large raw files as the core repository primitive. do not try to support every neuroscience package as a first-class node library. and do not promise full hipaa-grade regulated hosting before the operational controls, contracting, and support model actually exist. those cuts are what make the rest of the stack coherent. ³¹

for the two-year roadmap, the biggest upgrade is not “more features”; it is stronger platform boundaries. first, add openlineage-style lineage emission so that runs, inputs, outputs, datasets, and artefacts are traceable across services. second, harden multi-tenant execution with stronger sandboxing such as gvisor for selected classes of workloads. third, add nextflow interoperability for institutions that already have hpc-oriented scientific pipelines and want to import or export them cleanly. fourth, expand collaboration with comments, presence, and review flows around pipeline diffs and dataset snapshots. fifth, add richer execution modes: remote kernels for interactive exploratory work, and wasm-based plugins for lightweight validation or parsing steps. sixth, if repository search truly becomes a product differentiator, introduce opensearch with explicit indexing contracts rather than letting ad hoc search sprawl into the control plane. ³²

on the engineering organisation side, a monorepo is the better default for the first phase. the reason is coordination cost: ui node definitions, backend schema changes, workflow spec evolution, sdk ergonomics, docs, and example pipelines will move together constantly. splitting too early into many repositories usually raises release friction faster than it improves autonomy. the monorepo should still enforce package boundaries: `frontend`, `api`, `worker-images`, `python-sdk`, `schema`, `examples`, and `docs`. testing should not stop at unit tests. this domain needs golden bids and nwb fixtures, workflow snapshot tests, permission matrix tests, and true end-to-end runs against sample public datasets. the contributor model should then mirror the architecture: documented plugin interfaces for readers, node templates, and execution backends; docs-as-code; and sample extension repos once the core APIs settle. ³³

integration priorities should follow user value, not theoretical completeness. openneuro and dandi are the obvious first integrations because they map directly onto bids and nwb-centred public repository workflows. pennsieve matters because it proves demand for collaborative workspaces, metadata-rich scientific data management, and open api access. cloud-provider integration should remain deliberately thin at first: object storage, kubernetes, secrets, and optional identity connectors are enough. institution-specific adapters, specialised hpc schedulers, and fancy data movers belong on the roadmap once the base repository and execution model is stable. ³⁴

open questions and limitations

some important choices remain product-strategy questions more than technical ones. the biggest is whether the visual editor is mainly a no-code authoring surface, a documentation and review layer over code-defined pipelines, or both. that decision changes how much weight to give python export, round-

tripping fidelity, and compatibility with external pipeline definitions. the research above supports all three possibilities, but does not by itself choose among them. 35

a second unresolved issue is the exact trust model for hosted user code. containers are enough for many internal or lab deployments, but true multi-tenant hosted execution raises harder questions about sandbox escape risk, egress controls, secrets exposure, and kernel-style interactive sessions. the report therefore recommends containers first and stronger isolation later, but the correct timing depends on who the first paying or institutional users are. that is an operational and commercial question as much as an engineering one. 36

the last limitation is evidentiary. i prioritised official, primary, english-language sources and avoided filling gaps with weak secondary material. where a project's product documentation is strong but its licence or internal implementation detail was not freshly re-checked in those same docs, i treated the recommendation as an engineering judgement rather than pretending to have stronger evidence than i do. the main conclusions above are still high-confidence, but a procurement-grade build sheet would benefit from a final pass over individual repos, helm charts, and support policies before committing to a specific production rollout.

1 5 6 7 34 **OpenNeuro**

https://openneuro.org/?utm_source=chatgpt.com

2 8 14 **MNE — MNE 1.12.1 documentation**

https://mne.tools/stable/index.html?utm_source=chatgpt.com

3 18 22 **Quick Start - Argo Workflows - The workflow engine for Kubernetes**

https://argo-workflows.readthedocs.io/en/latest/quick-start/?utm_source=chatgpt.com

4 20 30 **Next.js Docs | Next.js**

https://nextjs.org/docs?utm_source=chatgpt.com

9 **Braindecode — Decode raw EEG, ECoG and MEG with deep learning — Braindecode 1.5.1 documentation**

https://braindecode.org/stable/index.html?utm_source=chatgpt.com

10 33 **Welcome to pybids's documentation! — PyBIDS 0.22.0 documentation**

https://bids-standard.github.io/pybids/?utm_source=chatgpt.com

11 **NWB for Python — PyNWB 3.1.2 documentation**

https://pynwb.readthedocs.io/?utm_source=chatgpt.com

12 **Welcome to the EEGLAB Wiki - EEGLAB Wiki**

https://eeglab.org/?utm_source=chatgpt.com

13 **Nilearn**

https://nilearn.github.io/stable/index.html?utm_source=chatgpt.com

15 **PostgreSQL: Documentation: 16: 8.14. JSON Types**

https://www.postgresql.org/docs/16/datatype-json.html?utm_source=chatgpt.com

16 27 29 **Cloud Object Storage – Amazon S3 – Amazon Web Services**

https://aws.amazon.com/s3/?utm_source=chatgpt.com

17 26 **Overview | Kubernetes**

https://kubernetes.io/docs/concepts/overview/?utm_source=chatgpt.com

19 containerd docs – containerd Overview

https://containerd.io/docs/?utm_source=chatgpt.com

21 About gRPC | gRPC

https://grpc.io/about/?utm_source=chatgpt.com

23 The Security Rule | HHS.gov

https://www.hhs.gov/hipaa/for-professionals/security/index.html?utm_source=chatgpt.com

24 SAML 2.0 Technical Overview | OASIS Standard

https://docs.oasis-open.org/security/saml/Post2.0/sstc-saml-tech-overview-2.0.html?utm_source=chatgpt.com

25 Role-Based Access Control - Auth0 Docs

https://auth0.com/docs/manage-users/access-control/rbac?utm_source=chatgpt.com

28 What is AWS Batch? - AWS Batch

https://docs.aws.amazon.com/batch/latest/userguide/what-is-batch.html?utm_source=chatgpt.com

31 PostgreSQL: Documentation: 15: Chapter 12. Full Text Search

https://www.postgresql.org/docs/15/textsearch.html?utm_source=chatgpt.com

32 About OpenLineage | OpenLineage

<https://openlineage.io/docs/1.39.0/>

35 Introduction - Prefect

https://docs.prefect.io/?utm_source=chatgpt.com

36 What is gVisor? - gVisor

https://gvisor.dev/docs/?utm_source=chatgpt.com